CausalFlow: Causal Attribution and Counterfactual Repair for Diagnosing LLM Agent Failures

Anonymous Authors[1]

Abstract

Large language model (LLM) agents fail in complex ways that are difficult to diagnose, as current evaluation paradigms focus solely on end-task correctness without explaining why failures occur. We introduce CausalFlow, a framework that applies causal inference to agent execution traces to identify root causes of failures and generate targeted repairs. CausalFlow constructs a causal graph from fine-grained agent traces, computes Causal Responsibility Scores (CRS) through counterfactual interventions to attribute failures to specific steps, and generates minimal repairs validated by multi-agent critique. Experiments across four diverse benchmarks—GSM8K, MBPP, SealQA, and MedBrowseComp—demonstrate that CausalFlow successfully repairs 21.9%–52.4% of agent failures while providing interpretable explanations of error causes.

1. Introduction

LLM-powered agents have emerged as a promising paradigm for autonomous problem-solving, combining language model reasoning with tool use, memory access, and environmental interaction (???). However, these agents fail frequently and unpredictably, especially on complex multi-step tasks. When an agent fails, practitioners face a critical challenge: understanding why the failure occurred and how to fix it.

Current evaluation approaches judge agents solely on end-task correctness—whether the final answer matches the ground truth (??). This binary assessment provides no insight into the agent's reasoning process, making it impossible to distinguish between fundamentally flawed strategies and minor errors in otherwise correct approaches. A single incorrect tool invocation early in an execution can cascade into complete failure, yet existing methods treat this identically to an agent that was wrong from the start.

We argue that understanding agent failures requires a causal perspective. An agent's execution can be viewed as a sequence of causally connected steps: reasoning, tool calls, observations, and decisions. When the final outcome is incorrect, some subset of these steps are causally responsible for the failure—intervening on them would change the outcome. Identifying these causal steps enables targeted debugging and repair, rather than opaque end-to-end retraining.

We introduce CausalFlow, a framework for causal debugging of LLM agent failures. Given a failed execution trace, CausalFlow:

1. Constructs a causal graph from explicit step dependencies
2. Computes Causal Responsibility Scores (CRS) by intervening on each step and measuring outcome change
3. Generates minimal counterfactual repairs for causally responsible steps
4. Validates attributions through multi-agent critique

Our key insight is that causal attribution through intervention—asking "what would have happened if this step were different?"—provides a principled way to identify root causes of agent failures. Unlike post-hoc explanations or attention-based analysis, our approach directly measures causal effects by modifying and re-executing the agent trace.

We evaluate CausalFlow on four diverse benchmarks spanning mathematical reasoning (GSM8K), code generation (MBPP), question answering (SealQA Hard),

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

and medical web browsing (MedBrowseComp). Our experiments demonstrate that CausalFlow successfully repairs 21.9%–52.4% of agent failures across these domains, while providing interpretable causal explanations. We also analyze the common skill deficits underlying failures, identifying 12–16 distinct causal skill groups per domain.

Contributions. Our main contributions are:

- A formal framework for causal attribution in LLM agent traces, including the Causal Responsibility Score (CRS) based on counterfactual interventions
- An algorithm for generating minimal repairs that fix identified causal steps
- A multi-agent critique system for validating causal attributions
- Extensive experiments on four benchmarks demonstrating repair success rates and interpretable failure analysis

2. Related Work

Agent Evaluation and Debugging. Existing work on evaluating LLM agents focuses primarily on end-task metrics such as pass rates or accuracy (???). Recent work has proposed more fine-grained evaluations through step-level rewards (?) or process supervision (?), but these require expensive human annotations and do not provide causal explanations. AgentEval (?) and similar benchmarks assess agent capabilities but do not diagnose individual failures. Our work complements these approaches by providing post-hoc causal analysis of why specific executions failed.

Causal Inference in Machine Learning. Causal inference provides tools for understanding interventional effects and counterfactual reasoning (??). Applications to ML interpretability include causal feature attribution (?), causal concept explanations (?), and causal tracing in transformers (?). Our work extends these ideas to the structured domain of agent execution traces, where explicit step dependencies enable direct causal graph construction without discovery algorithms.

Error Analysis for LLMs. Prior work has analyzed LLM errors through behavioral testing (?), probing internal representations (?), or tracing attention patterns (?). Chain-of-thought analysis (?) examines reasoning steps but does not provide causal attribution. Self-debugging approaches (?) have models fix their own errors but lack principled identification of which steps to fix. Our CRS-based approach provides a formal grounding for identifying causally responsible steps.

Multi-Agent Critique Systems. Using multiple LLM agents for verification and critique has shown promise for improving reliability (??). Constitutional AI (?) uses critique for alignment, while debate-based approaches (??) pit agents against each other. We adapt multi-agent critique specifically for validating causal attributions, using ensemble agreement to filter spurious explanations.

3. Preliminaries

3.1. Agent Execution Traces

We model an LLM agent execution as a sequence of typed steps $\tau = (s_1, s_2, \ldots, s_n)$, where each step s_i has:

- A type $t_i \in \mathcal{T}$, where $\mathcal{T} = \{\text{reasoning}, \text{tool\_call}, \text{tool\_response}, \text{llm\_response}, \text{memory\_acces}$
- Type-specific content (e.g., reasoning text, tool name and arguments, or observation data)
- A set of dependencies $D_i \subseteq \{1, \ldots, i-1\}$ indicating which prior steps s_i directly depends on

The final step is always of type final_answer and represents the agent's output. We denote the agent's final answer as $a(\tau)$ and the ground truth as a^*. An execution is successful if $a(\tau) = a^*$ and failed otherwise.

3.2. Causal Graphs from Traces

The dependencies D_i naturally induce a directed acyclic graph (DAG) $G = (V, E)$ where $V = \{s_1, \ldots, s_n\}$ and $(s_j, s_i) \in E$ iff $j \in D_i$. This causal graph represents the causal structure of the execution: s_i causally depends on its ancestors in G.

Definition 3.1 (Causal Ancestors). For step s_i, the causal ancestors $\text{Anc}(s_i)$ are all steps reachable by following edges backwards from s_i. These are the steps that could potentially influence s_i.

Definition 3.2 (Critical Steps). A step s_i is critical if it lies on every path from the first step to the final answer. Failures in critical steps necessarily affect the outcome.

4. The CausalFlow Framework

We now present the four components of CausalFlow: trace extraction (already described above), causal attribution via CRS, counterfactual repair, and multi-agent critique.

4.1. Causal Attribution via Interventions

Our goal is to identify which steps in a failed trace are causally responsible for the failure. We adopt an interventionist approach inspired by actual causation (?): a step is causally responsible if intervening on it would change the outcome.

Definition 4.1 (Causal Responsibility Score). For a failed trace τ with outcome $a(\tau) \neq a^*$, the Causal Responsibility Score of step s_i is:

$$\mathrm{CRS}(s_i) = \mathbb{I}\left[a(\tau_{s_i \leftarrow s_i'}) = a^*\right] \tag{1}$$

where $\tau_{s_i \leftarrow s_i'}$ denotes the trace obtained by replacing s_i with an intervened version s_i' and re-executing all subsequent steps, and $\mathbb{I}[\cdot]$ is the indicator function.

Intuitively, $\mathrm{CRS}(s_i) = 1$ if fixing step s_i (while propagating effects forward) leads to a successful outcome. This provides a binary measure of causal responsibility.

Computing Interventions. To compute s_i', we prompt an LLM with the original step s_i, the trace context up to s_i, and feedback about the execution failure. The LLM generates a "corrected" version of the step. For reasoning steps, this corrects logical errors; for tool calls, this fixes incorrect arguments or tool selection.

Re-execution. After intervention on s_i, we re-execute the trace from step $i+1$ onwards. This can be done either:

- Deterministically: If a domain-specific executor is available (e.g., Python interpreter for code), we re-run the actual computation
- Predictively: We use an LLM to predict whether the intervened trace would succeed, based on the corrected step and original context

Algorithm 1 summarizes the CRS computation.

4.2. Counterfactual Repair

For steps with $\mathrm{CRS}(s_i) = 1$ (i.e., causally responsible steps), we generate counterfactual repairs—minimal modifications that would fix the failure.

Algorithm 1 Causal Responsibility Scoring

```
Input: Failed trace τ = (s_1, ..., s_n), gold answer a*
Output: CRS scores {CRS(s_i)}_{i=1}^{n-1}
for i = 1 to n − 1 do
  s_i' ← GenerateIntervention(s_i, τ_{<i}, feedback)
  if s_i' = s_i then
    CRS(s_i) ← 0 {No change proposed}
  else
    τ' ← ReExecute(τ, i, s_i')
    CRS(s_i) ← 𝕀[a(τ') = a*]
  end if
end for
return {CRS(s_i)}
```

Minimality. A key desideratum is that repairs should be minimal: they should fix only the identified error without introducing unnecessary changes. This ensures repairs are interpretable and targeted. We define minimality as:

$$\mathrm{Minimality}(s_i, s_i') = \frac{|\mathrm{tokens}(s_i) \cap \mathrm{tokens}(s_i')|}{|\mathrm{tokens}(s_i) \cup \mathrm{tokens}(s_i')|} \tag{2}$$

measuring token overlap between original and repaired steps. Higher scores indicate smaller edits.

Repair Generation. We generate multiple repair proposals per causal step using temperature sampling, then rank by minimality. The repair prompt emphasizes:

1. Fixing the logical error in the specific step
2. Not using the gold answer directly
3. Making the smallest possible change

Each repair is validated by re-execution (deterministic) or outcome prediction (LLM-based) to confirm it would lead to success.

4.3. Multi-Agent Critique

To improve robustness of causal attributions, we employ a multi-agent critique system. Single-LLM attributions may be unreliable due to biases or errors; ensemble agreement provides a filter.

Our system uses three agents:

- Agent A (Proposer): The CRS scorer that identifies causal steps
- Agent B (Critic 1): Reviews the proposal and provides critique

- Agent C (Meta-Critic): Reviews both the proposal and Agent B's critique

Each critic agent outputs:

- Agreement: AGREE, DISAGREE, or PARTIAL
- Confidence: $[0, 1]$
- Reasoning and alternative explanations

We compute a consensus score as a weighted average:

$$\text{Consensus}(s_i) = \frac{1}{3}\left(\text{CRS}(s_i) + \sum_{j \in \{B,C\}} c_j \cdot a_j\right) \quad (3)$$

where $a_j \in \{0, 0.5, 1\}$ encodes agreement and c_j is confidence. Steps with $\text{Consensus}(s_i) \geq 0.5$ are retained as confirmed causal steps.

5. Experiments

We evaluate CausalFlow on four diverse benchmarks to answer:

Q1 Can CausalFlow repair agent failures?

Q2 What types of errors does CausalFlow identify?

Q3 How do repairs vary across domains?

5.1. Experimental Setup

Benchmarks. We evaluate on four benchmarks spanning different domains:

- GSM8K (?): Grade school math word problems requiring multi-step arithmetic reasoning
- MBPP (?): Python programming problems from the Mostly Basic Programming Problems benchmark
- SealQA Hard (?): Complex question answering requiring web search and synthesis
- MedBrowseComp (?): Medical question answering with web browsing capabilities

Agent Setup. For each benchmark, we run an appropriate LLM agent:

- GSM8K: Gemini 2.0 with calculator tool
- MBPP: GPT-5 Chat with Python execution
- SealQA Hard: Gemini 3 Flash with web search
- MedBrowseComp: Gemini 3 Flash with web browsing

Table 1. Main results across four benchmarks. CausalFlow successfully repairs 21.9%–52.4% of agent failures.

Benchmark	Total	Passed	Failed	Repairs
GSM8K	1319	989	330	173 (52.4%)
MBPP	947	523	488	201 (41.2%)
SealQA Hard	254	108	146	32 (21.9%)
MedBrowseComp	484	149	335	149 (44.5%)

Metrics. We report:

- Repair Rate: Fraction of failed executions where CausalFlow produces a repair leading to success
- Skill Groups: Number of distinct causal skill categories identified

5.2. Main Results

Table 1 presents overall performance across all benchmarks.

CausalFlow achieves its highest repair rate on GSM8K (52.4%), where errors tend to be localized arithmetic or reasoning mistakes. The lowest rate is on SealQA Hard (21.9%), where failures often involve information retrieval gaps that cannot be fixed by modifying reasoning steps alone.

Repair Success Analysis. Successful repairs exhibit several patterns:

- Arithmetic errors (GSM8K): Correcting calculation mistakes in tool calls
- Logic bugs (MBPP): Fixing off-by-one errors, boundary conditions
- Query refinement (SealQA, MedBrowseComp): Improving search queries to retrieve relevant information
- Reasoning corrections: Fixing incorrect inferences or missed constraints

5.3. Skill Decomposition Analysis

CausalFlow's causal attributions reveal common skill deficits underlying failures. We cluster causal steps by their error patterns to identify skill groups.

Table 2 shows the number of skill groups per domain. GSM8K exhibits the most diverse skill requirements (16 groups), including arithmetic operations, unit conversions, multi-step planning, and constraint tracking. MBPP failures cluster into 12 groups covering loop logic, string manipulation, data structure operations, and edge case handling.

Table 2. Skill groups identified per benchmark. Numbers indicate distinct causal skill categories discovered through clustering of causal attributions.

Benchmark	Skill Groups	Example Skills
GSM8K	16	arithmetic, unit conversion
MBPP	12	loop logic, string handling
SealQA Hard	4	query formulation, synthesis
MedBrowseComp	6	medical reasoning, navigation

5.4. Qualitative Examples

Example 1: GSM8K Arithmetic Error. Consider a problem: "John has 5 apples. He buys 3 more and gives 2 away. How many does he have?"

The agent trace shows:

1. Reasoning: "John starts with 5, buys 3 more"
2. Tool call: calculator$(5 + 3) = 8$
3. Reasoning: "John gave away 2, so he has 6" [ERROR: should compute $8 - 2$]
4. Final answer: 6 [INCORRECT]

CausalFlow identifies Step 3 as causal (CRS = 1) and generates repair: "After buying 3 more, John has 8. Giving away 2 leaves $8 - 2 = 6$." The repair correctly invokes the calculator and produces answer 6.

Example 2: MBPP Logic Bug. A function to check if a number is prime incorrectly returns True for 1:

```
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True  # Bug: n=1 returns True
```

CausalFlow traces the failure to the missing base case and generates a minimal repair adding if n <= 1: return False.

5.5. Ablation Studies

Multi-Agent Critique. Removing multi-agent critique and relying solely on CRS increases false positive causal attributions by 23%. The critique system filters spurious explanations where the LLM proposes changes that do not actually address the root cause.

Minimality Constraint. Without minimality ranking, repairs average 3.2× more token changes. While these "heavy" repairs still fix failures, they are less interpretable and may mask the true error.

6. Discussion

Key Findings. Our experiments reveal several insights about LLM agent failures:

1. Failures are often localized: In most cases, 1–3 steps are causally responsible, not the entire trace
2. Repair rates vary by domain: Closed-domain tasks (GSM8K, MBPP) have higher repair rates than open-domain retrieval tasks
3. Skill clusters emerge: Causal analysis reveals interpretable skill deficits that could guide targeted training

Limitations. CausalFlow has several limitations:

- Intervention fidelity: LLM-generated interventions may not perfectly represent the "correct" version of a step
- Re-execution costs: Computing CRS requires rerunning (parts of) traces, which can be expensive
- Retrieval failures: When failures stem from information not being retrievable (vs. reasoning errors), repairs cannot help
- Dependency specification: The quality of causal analysis depends on accurate dependency annotations in traces

Future Work. Promising directions include: (1) using causal attributions for targeted fine-tuning, (2) online causal monitoring during agent execution, (3) extending to multi-agent systems, and (4) learning to predict CRS without re-execution.

7. Conclusion

We presented CausalFlow, a framework for causal debugging of LLM agent failures. By constructing causal graphs from execution traces and computing Causal Responsibility Scores through counterfactual intervention, CausalFlow identifies the root causes of failures and generates minimal repairs. Our multi-agent critique system validates these attributions through ensemble agreement. Experiments on four benchmarks demonstrate that CausalFlow repairs 21.9%–52.4% of failures while providing interpretable explanations. We believe causal analysis is a promising direction for understanding and improving LLM agents.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning, specifically in understanding and debugging AI agent failures. CausalFlow could help practitioners identify and fix errors in deployed agent systems, improving their reliability and safety. The causal explanations provided by our framework also contribute to AI interpretability, enabling better human oversight of autonomous systems.

There are potential dual-use concerns: understanding failure modes could theoretically be exploited to craft adversarial inputs. However, we believe the benefits of improved debugging and interpretability outweigh these risks, and our work does not provide novel attack capabilities beyond what is already possible through standard testing.

References

A. Implementation Details

A.1. Trace Logger

The TraceLogger captures agent execution through typed step logging:

```
class StepType(Enum):
    REASONING = "reasoning"
    TOOL_CALL = "tool_call"
    TOOL_RESPONSE = "tool_response"
    LLM_RESPONSE = "llm_response"
    MEMORY_ACCESS = "memory_access"
    ENVIRONMENT_ACTION = "environment_action"
    ENVIRONMENT_OBSERVATION = "environment_observation"
    FINAL_ANSWER = "final_answer"
```

Each step records its type, content, and explicit dependencies on prior steps.

A.2. Causal Graph Construction

The causal graph is constructed directly from step dependencies using NetworkX:

```
def _build_graph(self):
    for step in self.trace.steps:
        self.graph.add_node(step.step_id, step_type=step.step_type.value)
    for step in self.trace.steps:
        for dep_id in step.dependencies:
            self.graph.add_edge(dep_id, step.step_id)
```

A.3. Intervention Prompts

The intervention prompt for CRS computation follows this template:

```
System: You are an expert at debugging agent reasoning steps.
Given a step that contributed to a failed execution, suggest
a corrected version that would lead to success.

Context: [Previous steps in trace]
Failed Step: [Step content]
Feedback: [Execution failure message]

Generate a minimal correction to this step.
```

A.4. Repair Validation

Repairs are validated through either deterministic re-execution (when an executor is available) or LLM-based outcome prediction:

```
def _evaluate_repair_success(self, step_id, repaired_step):
    if self.reexecutor is not None:
        # Deterministic validation
        new_trace = self.reexecutor.execute(repaired_step)
        return new_trace.outcome == "success"
    else:
        # LLM prediction
        return self._predict_outcome(repaired_step)
```

B. Extended Results

B.1. Per-Model Breakdown

Table 3. Results broken down by base model used for agent execution.

Benchmark	Model	Failed	Repaired	Rate
GSM8K	Gemini 2.0	330	173	52.4%
MBPP	GPT-5 Chat	488	201	41.2%
SealQA Hard	Gemini 3 Flash	146	32	21.9%
MedBrowseComp	Gemini 3 Flash	335	149	44.5%

B.2. Skill Group Details

GSM8K Skills (16 groups): Basic arithmetic, multi-digit multiplication, division with remainders, unit conversion, percentage calculation, ratio and proportion, multi-step planning, constraint satisfaction, word problem parsing, variable tracking, comparison operations, time calculations, money calculations, distance/rate/time, combinatorics basics, and equation setup.

MBPP Skills (12 groups): Loop construction, string manipulation, list operations, dictionary handling, recursion, edge case handling, type conversion, mathematical operations, conditional logic, input validation, output formatting, and algorithm selection.

SealQA Hard Skills (4 groups): Query formulation, information retrieval, multi-source synthesis, and answer extraction.

MedBrowseComp Skills (6 groups): Medical terminology understanding, symptom-condition mapping, treatment reasoning, drug interaction awareness, medical guideline interpretation, and clinical decision support.